

Software Design Methods and Tools

Lab 1 Report: Maven & the Binary Calculator

Course: ENGR 5520G **Student:** Rohan Muslekar **Student ID:** 101006689 **Date:** 2026-09-13

GitHub repository: <https://github.com/Rohan-Muslekar/sdmt-binarycalculator-a1> **Demonstration video:** <https://drive.google.com/file/d/1Bde5twvs23cTunPDfQLDkVVB4TNccZB/view>

1. Overview

The project is a Maven-managed Java application that stores unsigned binary numbers as strings and performs arithmetic and logical operations on them. The starter code provided a `Binary` class with a validating constructor, a `getValue()` accessor, and a static `add` method. The design assignment extended the class with three new static operations: **bitwise OR**, **bitwise AND**, and **multiplication**. Each takes two `Binary` operands and returns a new `Binary` result. The build also produces auto-generated Javadoc documentation and a runnable JAR with its dependencies bundled in.

Project coordinates:

Field	Value
groupId	com.ontariotechu.sofe3980U
artifactId	BinaryCalculator
version	1.0.0

2. Source code analysis

2.1 Representation

A `Binary` holds its value as a `String` field `number` (e.g. `"1010"`). The constructor validates the input, rejecting `null`, empty, and any string containing a character other than `'0'` / `'1'` by defaulting to `"0"`, and strips leading zeros so every value has a canonical form. This canonical form matters: the tests compare results with `getValue().equals(...)`, so `"0001010"` and `"1010"` must resolve to the same string.

2.2 Existing `add`

`add` walks both operands from their least-significant digit (rightmost character) toward the most significant, carrying as it goes. This is the standard schoolbook addition algorithm in base 2. The loop continues while either operand has digits left or a carry remains, which is what lets it handle operands of different lengths and a final carry that grows the result by one digit. The three new methods reuse this same right-to-left walking pattern.

2.3 New method: `or`

```
public static Binary or(Binary num1, Binary num2)
```

Bitwise OR must align the two numbers on their least-significant bit, exactly as `add` does. The method indexes both strings from the right; when one operand is shorter and its index runs past the start, that position contributes `0` (a leading-zero pad). Each output digit is `1` when either input digit is `1`. The result string is built by prepending each computed digit, then wrapped in a `new Binary(...)` so leading zeros are normalized.

Example: `10001000 OR 111000 = 10111000`.

2.4 New method: `and`

```
public static Binary and(Binary num1, Binary num2)
```

Structurally identical to `or`, differing only in the combining rule: an output digit is `1` only when both input digits are `1`. Positions past the end of the shorter operand contribute `0`, which correctly forces the AND of those positions to `0`. Because the result can collapse to all zeros, wrapping in `new Binary(...)` is what normalizes a value like `"000"` down to the canonical `"0"`.

Example: `10001000 AND 111000 = 1000`.

2.5 New method: `multiply`

```
public static Binary multiply(Binary num1, Binary num2)
```

Multiplication uses the shift-and-add algorithm, the binary equivalent of long multiplication. It keeps a running total (initially `0`) and a copy of `num1` that is shifted left one place on each iteration. Walking `num2` from its least-significant digit: whenever the current bit of `num2` is `1`, the current shifted value of `num1` is added into the total via the existing `add` method. Shifting left by one place is done by appending a `'0'` to the string, which doubles the value. This reuse of `add` keeps the new code small and leans on already-tested arithmetic.

Example: `10001000 (136) x 111000 (56) = 1110111000000 (7616)`.

3. Testing code analysis

Tests use **JUnit 4.11** with `assertTrue` on `getValue().equals(...)`. Each new operation has three test cases (the assignment minimum), and each case targets a distinct scenario rather than repeating the same shape:

Method	Test	Scenario covered
<code>or</code>	<code>orOfEqualLengthNumbers</code>	operands of equal length
	<code>orOfDifferentLengthNumbers</code>	shorter operand, leading-zero alignment
	<code>orOfTwoZeros</code>	zero identity case
<code>and</code>	<code>andOfEqualLengthNumbers</code>	operands of equal length
	<code>andWithNoCommonSetBits</code>	different lengths, result collapses to <code>0</code>

Method	Test	Scenario covered
	<code>andWithZeroOperand</code>	one operand is zero
<code>multiply</code>	<code>multiplyTwoNonZeroNumbers</code>	general case ($5 \times 3 = 15$)
	<code>multiplyByPowerOfTwo</code>	pure left-shift ($2 \times 4 = 8$)
	<code>multiplyByZero</code>	zero absorbing element

The different-length and zero cases are the ones most likely to catch bugs: the alignment logic and the canonicalization of an all-zero result are exactly where a naive implementation goes wrong. Test names follow an intention-revealing convention (each name states the scenario) consistent with the starter's constructor tests.

Combined with the six inherited constructor tests and five `add` tests, the suite runs **20 tests**, all passing.

4. Build configuration

The `pom.xml` was configured for three additional concerns beyond compilation:

- **Dependency management.** `joda-time 2.9.2` is declared as a runtime dependency; `App` uses `org.joda.time.LocalDateTime` to print the current time, demonstrating a third-party library on the classpath.
- **Executable JAR with dependencies.** The `maven-assembly-plugin` with the `jar-with-dependencies` descriptor bundles `joda-time` into a single runnable JAR (`BinaryCalculator-1.0.0-jar-with-dependencies.jar`), with `App` set as the manifest `Main-Class`.
- **Documentation.** The `maven-javadoc-plugin` and `maven-surefire-report-plugin` are registered under `<reporting>`, so `mvn site` produces browsable API docs (including the three new methods) and a test report.

5. Reproducing the results

All numbers in this report were produced with the following commands, run from the project root (`BinaryCalculator/`) with `JAVA_HOME` pointing at JDK 11 and Maven 3.9.9 on the `PATH`.

Run the test suite (produces the "Tests run: 20" figure):

```
mvn test
```

Build the fat JAR and run the application (produces the sample OR/AND/multiply output):

```
mvn clean package assembly:single
java -cp target/BinaryCalculator-1.0.0-jar-with-dependencies.jar com.ontariotechu.sofe3980U.App
```

Generate the Javadoc site and test report:

```
mvn site
# open target/site/index.html (API docs at target/site/apidocs/index.html)
```

Observed application output:

```
The current local time is: 12:46:15.694
First binary number is 10001000
Second binary number is 111000
Their summation is 11000000
Their bitwise OR is 10111000
Their bitwise AND is 1000
Their product is 1110111000000
```

Observed test result:

```
Tests run: 20, Failures: 0, Errors: 0, Skipped: 0
BUILD SUCCESS
```

6. Conclusion

The three required operations were implemented as static `Binary` methods reusing the constructor's canonicalization and the existing `add` routine, keeping the new code compact and consistent with the starter's style. Each operation is covered by three scenario-focused JUnit tests, the full suite of 20 tests passes, the application builds into a self-contained runnable JAR, and Javadoc documentation is generated for all methods including the new ones.